T16 Native Sandbox: Architecture

Technical Advanced

How Claude Code isolates its operations at the OS level

Principle: the OS as security perimeter

Since v2.1.0, Claude Code includes a native sandbox that isolates bash command execution at the kernel level. The core idea is to invert the usual logic: rather than validating each command with a user confirmation, Claude operates freely inside OS-enforced boundaries. The sandbox becomes the guardian, not the permissions system.

Activation: `/sandbox` in Claude Code (interactive menu).

OS primitives by platform

macOS: Seatbelt. Built into the system, no installation required. Uses the TrustedBSD Mandatory Access Control framework to filter system calls at the kernel level. Measure overhead with your own commands.

Before v2.1.309, sandboxed Bash commands ran the shell's startup files outside the sandbox on macOS and Linux; update to v2.1.309 or later.

Linux / WSL2: bubblewrap. Requires installing two packages.

```
sudo apt-get install bubblewrap socat   # Ubuntu/Debian
sudo dnf install bubblewrap socat       # Fedora
```

Creates an isolated namespace (mount, network, PID, IPC) for each executed command. Performance depends on the workload and platform.

WSL1: not supported. **Native Windows**: not supported; use WSL2.

Filesystem isolation

The default policy is asymmetric and intentional: read access across the entire system, write access includes the working directory, the per-user temporary directory and explicitly added directories, subject to protected paths and configured rules.

Access	Scope
Read	Entire computer (except deny rules)
Write	Working directory, temporary directory and added directories, subject to restrictions
Execute	Commands within the sandboxed environment

To restrict reading of sensitive directories (SSH, cloud credentials), use `permissions.deny` in `settings.json`:

```
{
  "permissions": {
    "deny": ["Read(~/.ssh/**)", "Read(~/.aws/**)", "Edit(~/.ssh/**)",
"Edit(~/.aws/**)" ]
  }
}
```

Network isolation

All network connections from sandboxed commands go through a SOCKS5 proxy running outside the sandbox. The proxy applies domain-based filtering without inspecting traffic content (no deep packet inspection).

Two modes via `sandbox.network.strictAllowlist`: permissive (default, a host outside `allowedDomains` raises a permission prompt) or strict (`strictAllowlist: true`, a host outside the list fails outright, no prompt).

```
{
  "sandbox": {
    "network": {
      "strictAllowlist": true,
      "allowedDomains": ["api.anthropic.com", "*.npmjs.org", "github.com"]
    }
  }
}
```

Two operating modes

Auto-allow: bash commands inside the sandbox are automatically approved. Commands incompatible with the sandbox (e.g., `docker`) trigger a fallback to the normal permissions flow. Recommended for day-to-day development.

Regular permissions: sandboxed commands follow the normal permission rules; existing allow rules still apply. Useful for high-security environments or untrusted codebases.

The escape hatch: `dangerouslyDisableSandbox`

Some tools are incompatible with sandboxing because they require access to specific system resources (`docker` via `/var/run/docker.sock`, `watchman` via filesystem watch APIs). Claude may retry with this parameter. The retry uses the normal permission flow, which depends on the active mode; setting an ask rule can require a prompt.

To completely disable the escape hatch in critical environments:

```
{
  "sandbox": {
    "allowUnsandboxedCommands": false,
    "failIfUnavailable": true
  }
}
```

`failIfUnavailable: true` (v2.1.83+): causes the session to fail if the sandbox cannot start, rather than continuing without isolation. Recommended for production environments.